React

npx parcel one.js

control + c for quit

1.What is .parcel-cache?

.parcel-cache is a cache folder created by Parcel.

It stores temporary build data so Parcel can:

- **start faster next time**
- **rebuild only changed files**
- **avoid re-processing everything again**

Real-life example

Think of .parcel-cache like YouTube buffering:

- **First time → slow (needs to load everything)**
- **Next time → fast (already cached)**

Same with Parcel.

What does it contain?

- **Transformed JS / JSX files**
- **Dependency graph info**
- **Hashes of files**
- **Build optimization data**

Nothing critical / nothing you write manually

Should you delete .parcel-cache?

Yes, it's safe to delete

Parcel will recreate it automatically.

Delete when:

- **weird build errors**
- **stale code loading**
- **Parcel behaving strangely**

rm -rf .parcel-cache

Should you push .parcel-cache to Git?

NEVER

Add to .gitignore:

.parcel-cache

2.What is the dist folder?

- **dist stands for distribution.**
- **It is the folder where all your final, ready-to-use files go.**
- **The browser runs the files from the dist folder.**
- **Production ready code will go inside it.**

Example:

You write code like this:

one.html

one.js

style.css

After running:

npx parcel build one.html

Parcel creates:

dist/

one.html

one.abc123.js (JS converted & optimized)

style.abc123.css (CSS optimized)

dist contains production-ready files that users see.

Difference from .parcel-cache

Folder	Purpose
.parcel-cache	Temporary files for faster builds
dist	Files ready for users / deployment

- Run npx parcel build one.html → creates dist folder
- Deploy the contents of dist to a server (Netlify, Vercel, GitHub Pages)
- Don’t commit .parcel-cache to Git, but dist can be deployed

dist = the final website folder that the browser uses and you can deploy

3.What is JSX? (very simple English)

JSX = JavaScript XML

JSX lets you write HTML-like code inside JavaScript.

Why do we need JSX?

Without JSX (pure React):

```
const element = React.createElement(
  "h1",
  { className: "title" },
  "Hello"
);
```

With JSX (easy & clean):

```
const element = <h1 className="title">Hello</h1>;
```

Same result
JSX is easier to read and write

Important thing to know

Browser does NOT understand JSX

So:

- JSX is converted to JavaScript
- Tools like Babel do this conversion

JSX → React.createElement()

JSX rules (very important)

1.One parent element

Wrong:

<h1>Hello</h1>

** World **

Correct:

** **

<h1>Hello</h1>

** World **

** **

2.Use className, not class

<h1 className="title">Hello</h1>

3.JavaScript inside { }

const name = "Vijay";

<h1>Hello {name}</h1>

4.JSX must be closed

****

**
**

JSX is optional (but recommended)

You can write React without JSX, but:

- **code becomes hard to read**
- **more lines**
- **more mistakes**

So almost everyone uses JSX.

JSX is a syntax that allows us to write HTML-like code inside JavaScript, which React converts into normal JavaScript.